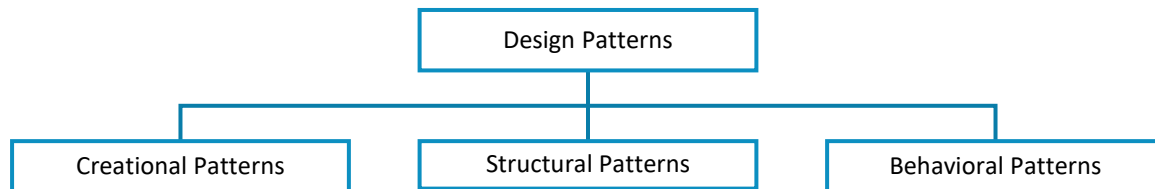


Design Patterns

⇒ What is Design Patterns?

Design patterns as proven recipes for solving common software problems. Just like a recipe tells us how to cook a dish efficiently, similarly a design pattern tells us how to structure our code efficiently.

Types of Design Patterns:



❖ Creational Patterns:

1) Singleton Pattern:

Singleton ensures that only one object of a class can exist in the entire application.

Why Do We Need It?

Imagine a bank application. If every module creates its own database connection, the server may become overloaded. Instead, all modules should share a single database connection.

Real-Life Example

Think of a country's Prime Minister. A country can have only one Prime Minister at a time.

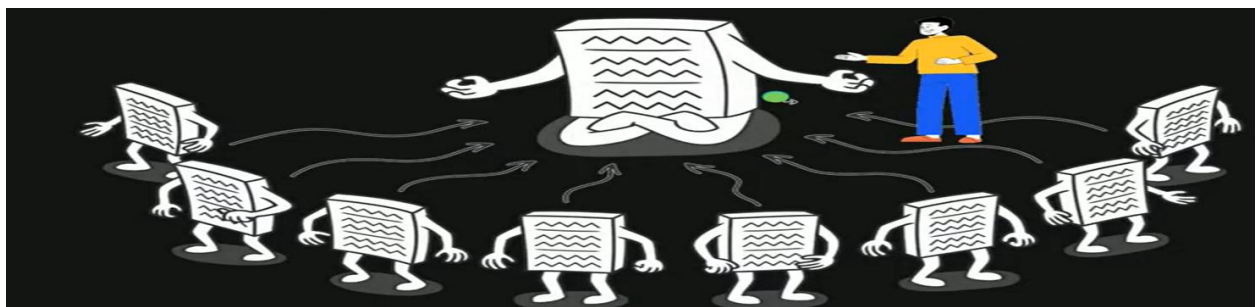
Similarly, Singleton allows only one object.

When to Use

- Only one instance is required.
- Multiple instances can create inconsistencies.

Examples:

- Database connection
- Logger
- Configuration settings
- Cache manager



How It Works?

The class stores its own instance. Whenever a new object is requested, it returns the existing one.

Implementation Strategy

1. Create a class variable to store the instance.
2. Override `__new__()`.
3. Return the existing object if already created.

Example

class Singleton:

```
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
```

```
obj1 = Singleton()
```

```
obj2 = Singleton()
```

```
print(obj1 is obj2)
```

2) Factory Pattern:

Factory creates objects without the user knowing the exact creation process.

Why Do We Need It?

Sometimes object creation depends on user input.

Real-Life Example:

Suppose We go to a restaurant and order a pizza. We don't cook the pizza Ourself. The chef decides how to prepare it. Here the restaurant acts as a factory.

When to Use

- Object type is decided at runtime.
- Many object types shares a common behavior.

Examples:

- Payment by Credit Card
- Payment by UPI
- Payment by PayPal



How It Works?

A factory method decides which class object to create.

Implementation Strategy

1. Define a common interface.
2. Create subclasses.
3. Build a factory class to instantiate objects.

Example

```
class Car:
```

```
    def drive(self):  
        print("Driving Car")
```

```
class Bike:
```

```
    def drive(self):  
        print("Riding Bike")
```

```
class VehicleFactory:
```

```
    @staticmethod  
    def get_vehicle(vehicle_type):  
        if vehicle_type == "car":  
            return Car()  
        elif vehicle_type == "bike":  
            return Bike()
```

```
vehicle = VehicleFactory.get_vehicle("car")
```

```
vehicle.drive()
```

3) Builder Pattern:

Builder constructs complex objects step by step.

Why Do We Need It?

Some objects contain many optional fields.

Real-Life Example:

When ordering a pizza:

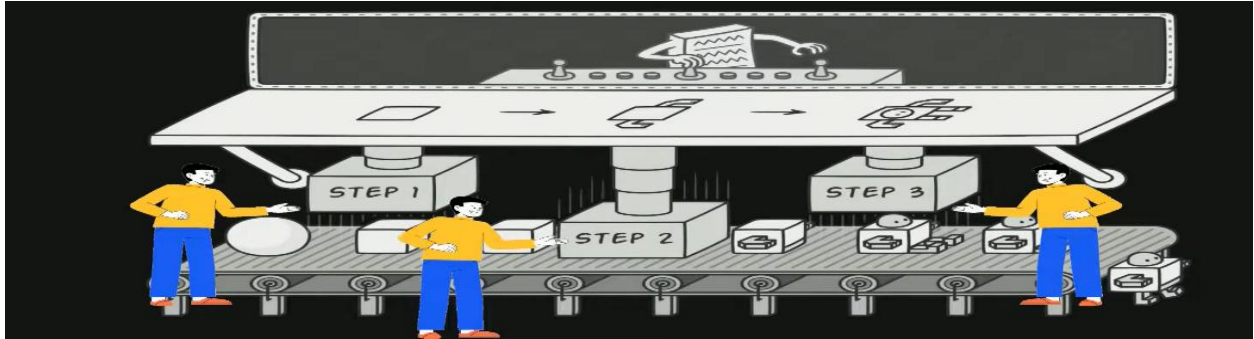
- Choose size
- Add cheese

- Add toppings

After selecting everything, the final pizza is prepared.

When to Use

- Objects have many optional parameters.
- Object construction is complex.



How It Works?

1. The builder creates the object step by step by adding required features.
2. After all features are added, it returns the final object.

Implementation Strategy

1. Create the target object.
2. Provide methods for setting attributes.
3. Return the final object using build().

Example

class Computer:

```
def __init__(self):
    self.cpu = None
    self.ram = None
```

class ComputerBuilder:

```
def __init__(self):
    self.computer = Computer()

def set_cpu(self, cpu):
    self.computer.cpu = cpu
    return self

def set_ram(self, ram):
    self.computer.ram = ram
    return self

def build(self):
    return self.computer
```

pc = (ComputerBuilder())

```
.set_cpu("Intel i7")
.set_ram("16GB")
.build()
print(pc.cpu, pc.ram)
```

❖ Structural Patterns:

4) Adapter Pattern:

Adapter allows incompatible classes to work together.

Why Do We Need It?

Old systems and new systems may have different interfaces.

Real-Life Example:

When traveling abroad, our charger may not fit the wall socket. In this case, we use an adapter.

The adapter connects two incompatible systems.

When to Use

- Integrating third-party libraries.
- Migrating legacy systems.



How It Works?

1. The adapter receives requests from the client.
2. It converts them into a format understood by another class.

Implementation Strategy

1. Identify incompatible interface.
2. Create adapter class.
3. Convert one interface into another.

Example

```
class OldPrinter:
```

```
    def print_text(self):
        print("Printing...")
```

```
class PrinterAdapter:
```

```
    def __init__(self, old_printer):
```

```
self.old_printer = old_printer
def print(self):
    self.old_printer.print_text()
printer = PrinterAdapter(OldPrinter())
printer.print()
```

5) Decorator Pattern:

Decorator adds new functionality without changing original code.

Why Do We Need It?

To add behavior dynamically.

Real-Life Example:

We buy coffee.

Base coffee = ₹50

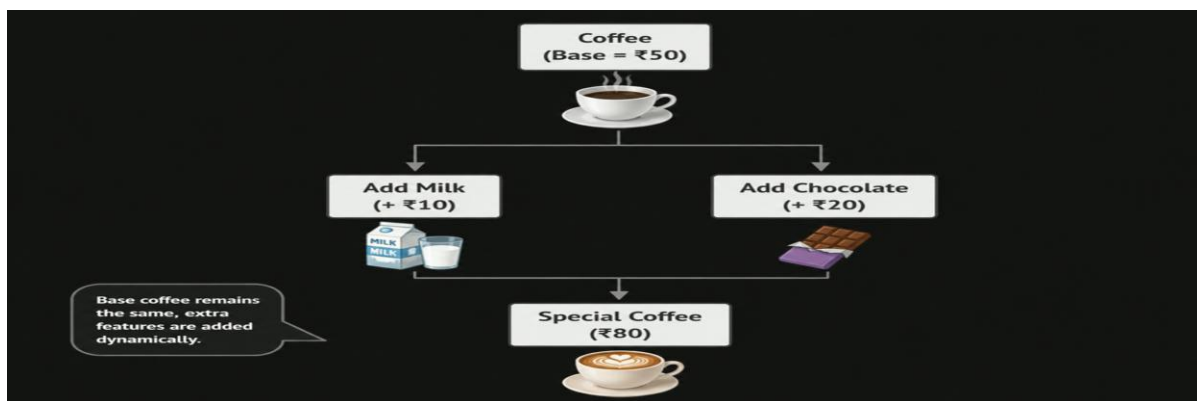
Add:

- Milk = +₹10
- Chocolate = +₹20

The coffee remains coffee, but extra features are added.

When to Use

- Logging
- Authentication
- Caching



How It Works?

1. The decorator wraps the original object or function.
2. It adds extra behavior before or after execution.

Implementation Strategy

1. Wrap the original function/object.
2. Add extra behavior before or after execution.

Example

```
def logger(func):  
    def wrapper():  
        print("Logging Started")  
        func()  
        print("Logging Ended")  
    return wrapper  
@logger  
def greet():  
    print("Hello")  
greet()
```

6) Façade Pattern:

Facade provides a simple interface to a complex system.

Why Do We Need It?

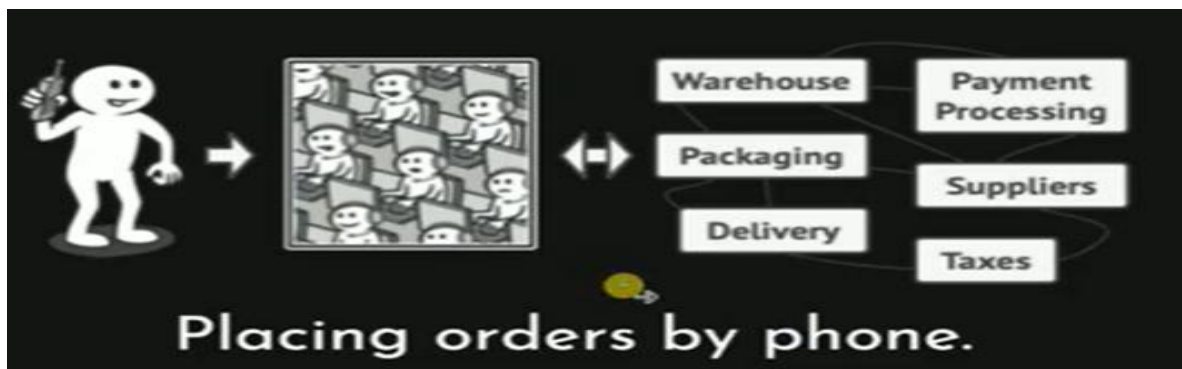
Complex systems have many modules. Facade provides a single interface to access them easily.

Real-Life Example:

When we place an order on Amazon, we only click "Place Order". Internally, payment, packaging, and delivery happen automatically. The order system acts as a Facade.

When to Use

- A system contains many modules.
- Users need a simple API.



How It Works

The Facade class communicates with multiple subsystems and performs all operations for the user.

Implementation Strategy

1. Create subsystem classes.
2. Build a facade class that coordinates them.

Example

```
class Payment:
```

```
    def process(self):  
        print("Payment done")
```

```
class Delivery:
```

```
    def deliver(self):  
        print("Product delivered")
```

```
class OrderFacade:
```

```
    def __init__(self):  
        self.payment = Payment()  
        self.delivery = Delivery()  
  
    def place_order(self):  
        self.payment.process()  
        self.delivery.deliver()
```

```
order = OrderFacade()
```

```
order.place_order()
```

❖ Behavioral Patterns:

7) Observer Pattern:

Observer automatically informs multiple objects when something changes.

Why Do We Need It?

One object changes → many objects should know.

Real-Life Example:

We all subscribe to a YouTube channel. When the creator uploads a video, all subscribers get notified.

When to Use

- Notification systems
- Event handling & Stock market apps



How It Works?

- Observers subscribe to the subject.
- When the subject changes, all observers are notified automatically.

Implementation Strategy

1. Subject maintains observer list.
2. Observers register themselves.
3. Notify all observers on changes.

Example

```
class Observer:
    def update(self, msg):
        print(msg)
class Subject:
    def __init__(self):
        self.observers = []
    def attach(self, observer):
        self.observers.append(observer)
    def notify(self, msg):
        for observer in self.observers:
            observer.update(msg)
sub = Subject()
sub.attach(Observer())
sub.attach(Observer())
sub.notify("Price Changed")
```

8) Strategy Pattern:

Strategy allows changing algorithms during runtime.

Why Do We Need It?

Different algorithms solve the same problem.

Real-Life Example:

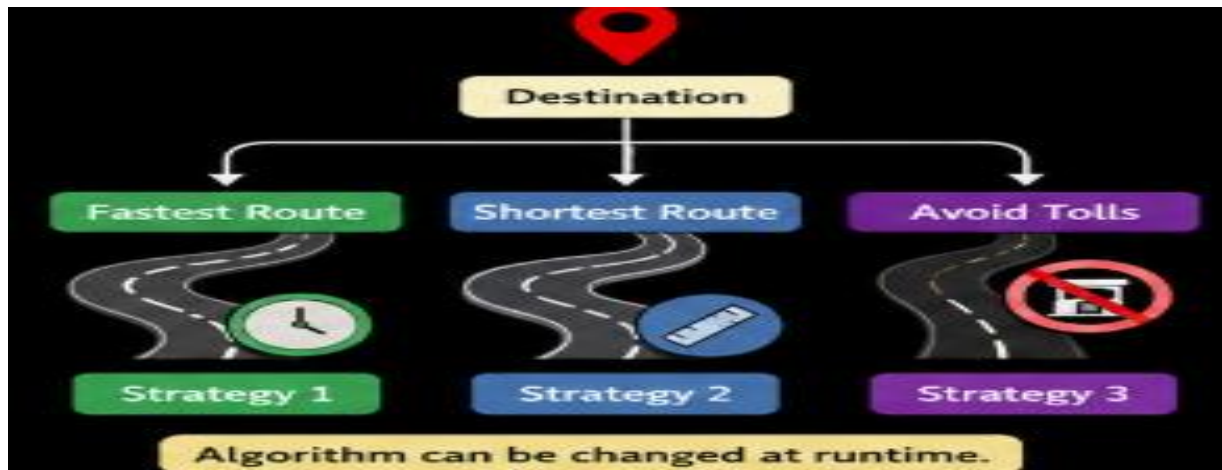
Google Maps provides multiple routes:

- Fastest route
- Shortest route
- Avoid tolls

The route selection algorithm changes dynamically.

When to Use

- Multiple algorithms solve the same problem.
- Algorithm changes at runtime.



Examples:

- Sorting algorithms
- Payment methods
- Compression algorithms

How It Works?

- Multiple algorithms are implemented as separate strategies.
- The client selects and changes the strategy at runtime.

Implementation Strategy

1. Define a strategy interface.
2. Create multiple strategies.
3. Inject strategy into the client.

Example

class Addition:

```
def execute(self, a, b):  
    return a + b
```

class Multiplication:

```
def execute(self, a, b):  
    return a * b
```

class Calculator:

```
def __init__(self, strategy):  
    self.strategy = strategy  
  
def calculate(self, a, b):  
    return self.strategy.execute(a, b)
```

```
calc = Calculator(Addition())  
print(calc.calculate(2, 3))  
calc.strategy = Multiplication()  
print(calc.calculate(2, 3))
```

9) Command Pattern:

Command converts a request into an object.

Real-Life Example:

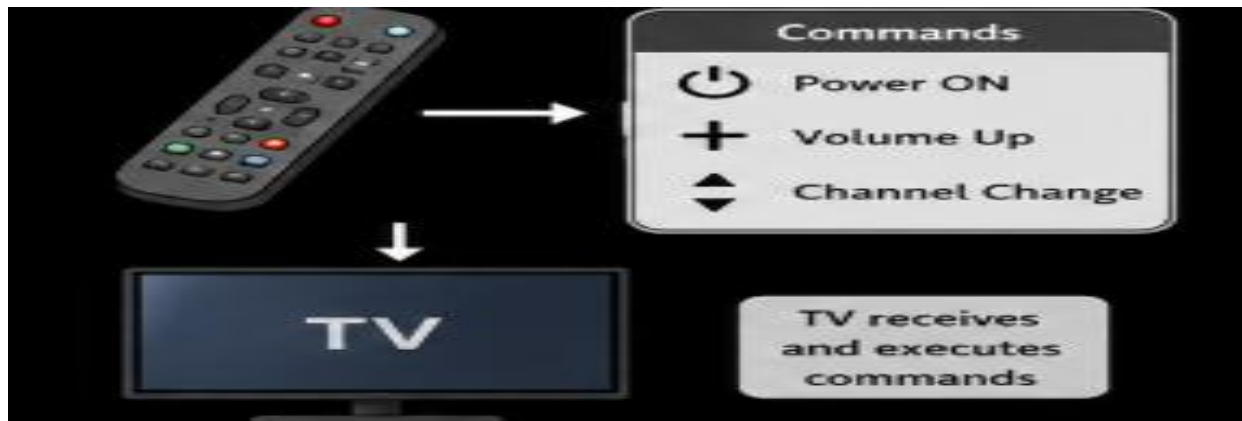
A TV remote stores commands like:

- Power ON
- Volume Up
- Channel Change

The remote doesn't know how the TV works internally. It simply sends commands.

When to Use

- Undo/Redo
- Task queues
- GUI buttons



How It Works?

- A request is converted into a command object.
- The receiver executes the command when invoked.

Implementation Strategy

1. Create command interface.
2. Implement concrete commands.
3. Execute through invoker.

Example

```
class Light:  
    def on(self):  
        print("Light ON")
```

```
class LightCommand:
    def __init__(self, light):
        self.light = light
    def execute(self):
        self.light.on()

light = Light()
command = LightCommand(light)
command.execute()
```

10) Template Method Pattern:

Defines a fixed workflow while allowing some steps to change.

Why Do We Need It?

When most steps are common, but some steps differ.

Examples:

- Data processing pipelines
- Report generation

Real-Life Example:

Making tea:

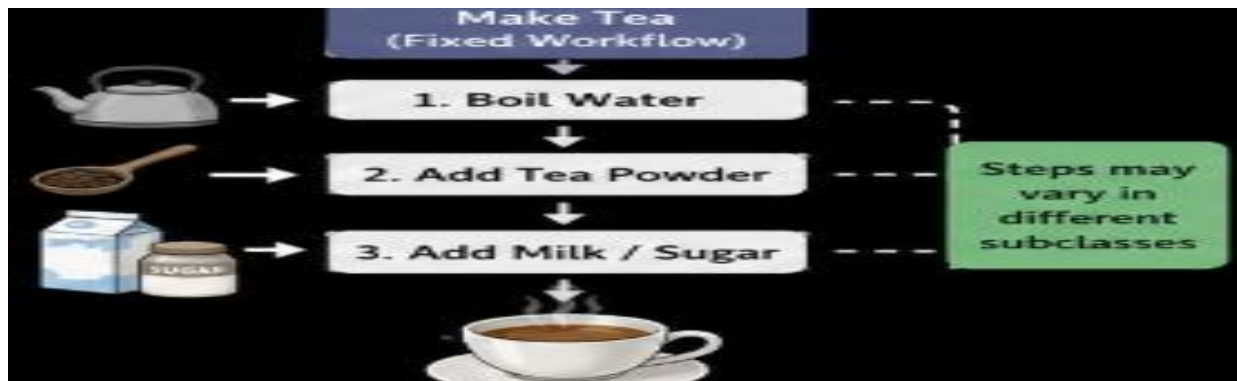
Steps:

1. Boil water
2. Add tea powder
3. Add milk

The overall process remains fixed, but ingredients may vary.

When to Use

- Workflow remains fixed.
- Some steps vary.



How It Works?

- The base class defines the fixed workflow.
- Subclasses implement or customize specific steps.

Implementation Strategy

1. Define template method.
2. Declare abstract methods.
3. Subclasses implement variable steps.

Example

```
from abc import ABC, abstractmethod
```

```
class DataProcessor(ABC):
```

```
    def process(self):
```

```
        self.read()
```

```
        self.analyze()
```

```
    @abstractmethod
```

```
    def read(self):
```

```
        pass
```

```
    @abstractmethod
```

```
    def analyze(self):
```

```
        pass
```

```
class CSVProcessor(DataProcessor):
```

```
    def read(self):
```

```
        print("Reading CSV")
```

```
    def analyze(self):
```

```
        print("Analyzing CSV")
```

```
csv = CSVProcessor()
```

```
csv.process()
```